

Hierarchical Generation of Synthetic Distributed Traces for Downstream Learning

Sneh Patel
Brock University
St. Catharines, ON, Canada
sp18oo@brocku.ca

Yuvraj Sehgal
Brock University
St. Catharines, ON, Canada
ys19rk@brocku.ca

Mahsa Panahandeh
University of Ottawa
Ottawa, ON, Canada
mpanahan@uottawa.ca

Naser Ezzati-Jivan
Brock University
St. Catharines, ON, Canada
nezzatijivan@brocku.ca

Francois Tetreau
Ciena Corporation
Ottawa, ON, Canada
ftetreau@ciena.com

Abstract

Distributed traces are a critical foundation for performance debugging, anomaly detection, and reliability engineering in modern microservice systems. In industrial settings, however, collecting and sharing large volumes of production traces are expensive and constrained by privacy and compliance requirements, which limits their availability for data-driven analysis and machine learning. This paper presents a hierarchical Variational Autoencoder (VAE) framework for generating synthetic distributed traces represented as directed acyclic graphs. The approach jointly models global execution structure and local span-level behavior, enabling the synthesis of realistic execution patterns that reflect real system behavior. We evaluate the fidelity and utility of the generated traces using clustering, dimensionality reduction, discriminative classification, and train-on-synthetic-test-on-real experiments on two microservice benchmarks. The results show strong overlap between real and synthetic trace distributions and demonstrate that models trained on synthetic traces generalize effectively to real traces. Augmenting synthetic data with a small fraction of real traces yields near-perfect downstream performance. These findings indicate that synthetic traces can reduce reliance on sensitive production data while supporting practical analysis and learning tasks in industrial AIOps and observability pipelines.

ACM Reference Format:

Sneh Patel, Yuvraj Sehgal, Mahsa Panahandeh, Naser Ezzati-Jivan, and Francois Tetreau. 2026. Hierarchical Generation of Synthetic Distributed Traces for Downstream Learning. In *Proceedings of Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE '26)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE '26, Montreal, QC, Canada

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern microservice-based software systems rely on distributed tracing to understand end-to-end request execution across services [15]. In industrial environments, traces are a primary source of evidence for performance debugging, anomaly detection, and reliability engineering, and increasingly serve as input to learning-based AIOps pipelines [20]. At the same time, trace collection and sharing are constrained by privacy, storage cost, and retention policies, which limits the amount of data that can be used for training and evaluation.

Prior work on trace-driven analysis and production observability [1, 5] shows that large organizations may record substantial volumes of traces, but this does not necessarily translate into effective training data. In practice, including in our discussions with engineers at Ciena, traces are often sampled, truncated, or retained for short periods. This biases repositories toward frequent, benign executions and underrepresents rare or complex behaviors. These long-tail executions are often the most informative for debugging and robustness, yet they are the hardest to capture, keep, and share across teams.

Synthetic distributed traces offer a way to supplement or partially replace real data under these constraints. Existing approaches, however, often rely on rule-based models, handcrafted templates, or sequential representations that do not capture the structural complexity of distributed executions or provide clear guidance on when synthetic traces can be used safely in downstream pipelines, especially for graph-based learning tasks.

Distributed traces naturally form directed acyclic graphs (DAGs), where nodes correspond to spans and edges represent parent-child execution relationships [21]. These graphs encode both global execution structure, such as request flow and service interaction patterns, and local span-level behavior, such as operation semantics and timing. Preserving both levels is important for generating synthetic traces that are structurally valid, semantically meaningful, and useful for analysis in operational settings.

Variational Autoencoders (VAEs) [9, 12] provide a probabilistic framework for learning latent representations of complex data distributions and generating new samples. By modeling distributions rather than deterministic mappings, VAEs are well suited to capturing variability in distributed system behavior. Applying VAEs directly to distributed trace graphs, however, requires handling global execution patterns and local span attributes in a unified way.

We address this problem with a hierarchical VAE framework for generating synthetic distributed traces represented as graphs. The model learns latent representations at two levels: a graph-level latent that captures global execution patterns, and node-level latents that model fine-grained span variability. This design reflects how real systems behave and enables the generation of diverse yet coherent traces. The framework supports fixed-size traces, which isolate structural fidelity, and variable-size traces, which introduce controlled execution variability.

Our evaluation emphasizes downstream utility rather than reconstruction accuracy alone. We examine whether models trained on synthetic traces generalize to real traces, whether synthetic data can replace or supplement limited real data, and how structural variability affects robustness. The evaluation follows established synthetic data practices, including train-on-synthetic-test-on-real experiments, clustering analysis, and discriminative testing, with a focus on task-relevant fidelity.

This paper makes three contributions:

- It articulates an industry-driven need for synthetic distributed traces, grounded in practical constraints observed with engineers at Ciena, including data scarcity, privacy limitations, and insufficient trace diversity for training and validating learning-based observability tools.
- It presents a hierarchical, graph-based generative framework that models distributed traces as directed acyclic graphs and separates global execution structure from local span-level behavior, with support for both fixed-size and variable-size generation.
- It provides a deployment-oriented empirical evaluation that goes beyond reconstruction accuracy to assess downstream utility, including train-on-synthetic-test-on-real performance, hybrid training with limited real data, and structural similarity and separability analyses, leading to guidance for industrial observability and AIOps pipelines.

This study is guided by the following research questions:

- RQ1) How accurately do synthetic traces preserve the structural and feature-level properties of real distributed traces?**
- RQ2) To what extent can synthetic traces replace or supplement real trace data in downstream analysis tasks?**
- RQ3) How does trace variability (fixed-size versus variable-size graphs) affect generation fidelity and robustness?**
- RQ4) How similar are synthetic and real traces in the joint feature space, and how easily can they be distinguished using common analysis techniques?**

2 Related Work

Analyzing system behavior for performance anomalies and failures is a fundamental task in the development and operation of distributed systems. In industrial settings, however, trace-driven analysis is often constrained by limited data availability, privacy concerns, and the cost of continuous instrumentation. These constraints have motivated a growing body of work on reducing observability requirements and generating synthetic traces for analysis and evaluation.

Ernst and Tai [3] propose an offline, model-driven approach for generating distributed traces to evaluate microservice observability without relying on runtime instrumentation. Their method models microservices as behavioral execution paths and enables reproducible benchmarking of observability tools. While effective for controlled evaluation, this approach relies on rule-based modeling and does not learn execution structure directly from real trace data, limiting its ability to capture the variability and complexity observed in production systems.

Janecek et al. [10] investigate diffusion-based generative models for reconstructing missing events in execution traces. Their work addresses trace incompleteness by performing sequence-level data imputation and demonstrates that diffusion models can effectively recover lost system call events under varying degrees of missingness. In contrast to trace reconstruction, our work focuses on generating complete synthetic distributed traces and explicitly models global execution structure using graph representations.

Synthetic data generation has also been explored in adjacent domains to address privacy and data scarcity. Giuseppe et al. [7] propose a Conditional Variational Autoencoder (CVAE) for generating privacy-preserving network traffic traces to train network intrusion detection systems. Their evaluation employs statistical similarity measures and train-on-synthetic-test-on-real (TSTR) performance, closely aligning with our evaluation methodology. However, their work targets flat network traffic data and security classification tasks, whereas we focus on distributed traces represented as directed acyclic graphs and emphasize structural fidelity and downstream utility in system-level analysis.

More recently, learning-based approaches have been proposed to generate synthetic traces across different system layers. Pang and Kant [18] introduce a GAN-based framework for synthesizing storage access traces that preserve coarse-grained spatio-temporal characteristics for workload augmentation. Kim et al. [11] explore the use of large language models (LLMs) to generate synthetic microservice call graphs by recursively constructing traces that satisfy structural and temporal constraints. While both approaches demonstrate the potential of synthetic traces to replace or supplement real data, they differ fundamentally in modeling assumptions and focus. Our work adopts a hierarchical VAE framework that directly learns graph-structured latent representations from real distributed traces and evaluates not only generation fidelity but also practical downstream utility.

Lu et al. [16] provide a comprehensive survey of machine learning-based synthetic data generation techniques, covering VAEs, GANs, diffusion models, and language models across multiple application domains. The survey highlights synthetic data as a practical response to privacy, cost, and regulatory constraints, and discusses common evaluation practices such as statistical similarity and TSTR performance. While prior work has primarily focused on tabular, image, text, and time-series data, our study targets graph-structured distributed traces and investigates how synthetic data supports trace-driven analysis in operational settings.

In summary, existing work has demonstrated the feasibility of synthetic trace generation and reconstruction across multiple domains. However, there remains a gap in understanding how graph-structured synthetic distributed traces can be generated from real execution data and used reliably in downstream analysis pipelines.

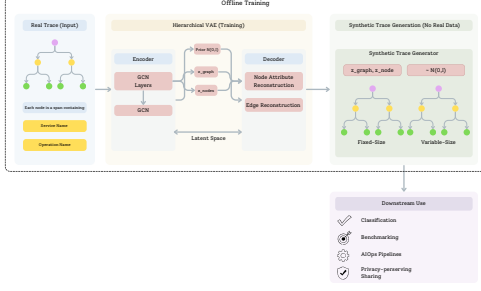


Figure 1: Overview of the hierarchical synthetic trace generation framework

This work addresses that gap by combining hierarchical generative modeling with an evaluation centered on practical utility and deployment-relevant trade-offs.

3 Methodology

This section describes the design and implementation of our synthetic distributed trace generation framework. The methodology is organized around the system components and design decisions required to generate realistic, graph-structured traces from real execution data. We first describe the datasets and graph representation used in our study, followed by the hierarchical encoder–decoder architecture and training objective. We then explain how synthetic traces are generated under different structural constraints.

3.1 Overview of the Framework

Figure 1 presents an overview of the proposed synthetic distributed trace generation framework. The framework is organized as a two-stage pipeline consisting of an offline training stage and a synthetic trace generation stage, reflecting practical constraints observed in industrial observability workflows.

During the offline training stage, real distributed traces are first converted into graph representations that capture both execution structure and span-level attributes. Each trace is modeled as a directed acyclic graph (DAG), where nodes correspond to spans and edges represent parent–child execution relationships. These graphs are used to train a hierarchical Variational Autoencoder (VAE) that learns latent representations at two levels: a graph-level latent encoding global execution structure, and node-level latents capturing fine-grained operational variability.

Once training converges, the encoder is no longer required. In the generation stage, synthetic traces are produced by sampling from the learned latent distributions and decoding these samples into complete trace graphs. This design cleanly separates representation learning from trace synthesis, enabling synthetic traces to be generated offline without access to real execution data.

From an engineering perspective, this separation has two important implications. First, synthetic trace generation can be integrated into existing observability pipelines as an offline preprocessing or

data augmentation step, without modifying live tracing infrastructure or instrumentation. Second, synthetic traces can be safely shared across teams or organizations, as generation does not depend on replaying or exposing real execution instances.

3.2 Trace Representation and Datasets

We consider a distributed trace dataset as a collection of execution traces generated by a microservice-based system under realistic workloads. Formally, let $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$ denote a set of distributed traces, where each trace T_i corresponds to the execution of a single request across multiple services.

Each trace T_i is represented as a directed acyclic graph (DAG) $G_i = (V_i, E_i)$, where V_i denotes the set of spans (nodes) and $E_i \subseteq V_i \times V_i$ denotes parent–child execution relationships between spans. The acyclic structure reflects the causal execution order imposed by distributed tracing systems. Each node $v \in V_i$ is associated with a feature vector

$$\mathbf{x}_v = (s_v, o_v, d_v),$$

where s_v denotes the service identifier, o_v denotes the operation identifier, and d_v denotes the execution duration of the span. This representation captures both structural dependencies and span-level semantics required for trace-driven analysis.

While the proposed framework is agnostic to the specific system and workload, we instantiate this formulation using two widely adopted microservice benchmarks introduced by Lee et al. [14]. These benchmarks provide realistic distributed trace data collected under controlled yet representative workloads and have been used extensively in prior work.

The first system, *SocialNetwork* (SN) [6], models a broadcast-style social networking application implemented using 21 microservices. The second system, *TrainTicket* (TT) [22], represents an online railway ticketing platform consisting of 41 microservices supporting ticket search, booking, and payment workflows. Both systems exhibit heterogeneous service interactions, varying execution depths, and skewed duration distributions that are characteristic of production microservice deployments.

From the original datasets, we select 1,244 traces from each system. The SN dataset contains 11,649 spans, with an average of 9.36 spans per trace (range 3–30), spanning 12 unique services and 59 unique operations. The TT dataset contains 7,912 spans, with an average of 6.36 spans per trace (range 1–23), spanning 13 unique services and 16 unique operations. Span durations range from milliseconds to several days, reflecting realistic long-tailed execution behavior.

For both datasets, traces are preprocessed by extracting parent–child relationships using the provided `parent_id` field and converting each trace into its corresponding DAG representation. This preprocessing step is independent of the proposed generative model and applies to any distributed tracing system that records causal span relationships and basic span-level attributes.

3.3 Hierarchical Encoder–Decoder Architecture

The proposed synthetic trace generation framework is built around a hierarchical Variational Autoencoder (VAE) that explicitly models distributed traces at multiple levels of abstraction. This design reflects the structure of distributed executions, which exhibit both

global request-level behavior and local span-level variability. Modeling these aspects jointly is essential for generating synthetic traces that are structurally valid and useful for downstream analysis.

Let $G = (V, E)$ denote a distributed trace graph, where V is the set of spans and E represents parent–child execution dependencies. Each node $v \in V$ is associated with a feature vector $\mathbf{x}_v = (s_v, o_v, d_v)$ encoding service identity, operation identity, and execution duration. The goal of the hierarchical VAE is to learn a latent variable model that captures both global execution structure and local operational behavior from such graphs.

To this end, the architecture decomposes trace modeling into two interacting components: a graph-level encoder–decoder that captures global execution patterns, and a node-level encoder–decoder that models fine-grained span-level variability conditioned on graph context. This separation enables the model to disentangle system-wide execution characteristics from local operational effects, while still allowing information to flow between levels during encoding and decoding.

This hierarchical design enables multi-granular representation learning beyond conventional graph autoencoders, allowing the model to disentangle global execution structure from local operational variability. Prior work has shown that hierarchical latent variable models improve expressiveness and representation quality for complex, structured data [17, 19].

3.3.1 Encoder. The encoder operates on graph-structured traces and maps each input trace G to a set of latent representations. Node features are first embedded using learnable embeddings for categorical attributes and a numerical encoder for span durations. These node embeddings are then propagated through multiple Graph Convolutional Network (GCN) layers [13], enabling information exchange along execution dependencies. This message-passing process allows each node representation to incorporate contextual information from related spans, reflecting the causal structure of distributed executions.

Formally, let $\mathbf{h}_v^{(0)}$ denote the initial embedding of node v . After L GCN layers, each node obtains a contextualized representation $\mathbf{h}_v^{(L)}$ that encodes both local attributes and structural context. These node representations are aggregated to produce a graph-level representation $\mathbf{h}_G$ using a permutation-invariant pooling operation.

The encoder produces three outputs. First, a graph-level latent variable z_G is parameterized by a variational posterior $q(z_G | G)$ with parameters $(\mu_G, \log \sigma_G^2)$ derived from $\mathbf{h}_G$. Second, each node v is associated with a node-level latent variable z_v , parameterized by $q(z_v | G)$ with parameters $(\mu_v, \log \sigma_v^2)$ derived from $\mathbf{h}_v^{(L)}$. Third, the deterministic node representations $\mathbf{h}_v^{(L)}$ are retained as auxiliary structural embeddings that are passed directly to the decoder.

Latent variables are sampled using the reparameterization trick, enabling stochastic generation while preserving end-to-end differentiability. The hierarchical separation between z_G and $\{z_v\}$ allows the encoder to model global execution structure independently from local operational variability.

3.3.2 Decoder. The decoder reconstructs node attributes and execution structure by conditioning on both graph-level and node-level latent variables. For each node v , the decoder receives three sources of information: the global latent z_G , the node-specific latent z_v ,

and the deterministic encoder embedding $\mathbf{h}_v^{(L)}$. These representations are fused to form a decoded node representation that balances global consistency, local stochastic variation, and precise structural context.

From the decoded node representations, the decoder predicts service labels, operation labels, and span durations. Categorical attributes are reconstructed using classification heads, while durations are reconstructed using a regression head. This multi-task formulation enables the model to jointly capture semantic and performance-related span characteristics.

To reconstruct execution structure, the decoder predicts directed edges between node pairs. Edge prediction is performed over ordered node pairs and constrained to respect the directed acyclic nature of distributed traces. Specifically, edge logits are produced for candidate parent–child relationships and later thresholded to form the reconstructed execution graph. Although this formulation has quadratic complexity in the number of nodes, it remains practical given the moderate trace sizes typically observed in real systems.

This encoder–decoder design allows the model to generate complete synthetic traces that preserve both execution structure and span-level semantics, while remaining agnostic to the specific system or workload from which the traces originate.

3.4 Training Objective

The hierarchical VAE is trained end-to-end using a multi-component objective that balances accurate reconstruction of distributed traces with structured regularization of the latent space. The objective is designed to ensure that the learned latent representations are expressive enough to capture realistic execution behavior, while remaining sufficiently constrained to support stable and diverse trace generation.

Given a distributed trace graph $G = (V, E)$, the training objective consists of three components: node attribute reconstruction, execution structure reconstruction, and latent regularization. Each component targets a distinct aspect of trace realism and utility.

3.4.1 Node Attribute Reconstruction. Node-level reconstruction focuses on recovering span semantics and performance characteristics. For each node $v \in V$ with feature vector $\mathbf{x}_v = (s_v, o_v, d_v)$, the decoder predicts service labels, operation labels, and execution durations.

The node reconstruction loss is defined as:

$$\mathcal{L}_{node} = \mathcal{L}_{service} + \mathcal{L}_{operation} + \mathcal{L}_{duration}, \quad (1)$$

where service and operation labels are optimized using weighted cross-entropy losses to mitigate class imbalance, and span durations are optimized using a regression loss (mean squared error). This formulation ensures that both semantic correctness and timing behavior are preserved in reconstructed traces.

3.4.2 Execution Structure Reconstruction. To recover execution dependencies, the decoder predicts directed edges between ordered node pairs. Since distributed traces form directed acyclic graphs, edge prediction is constrained to respect causal ordering.

Let $\hat{E}$ denote the set of predicted edges and E the ground-truth edge set. The edge reconstruction loss is defined using binary cross-entropy over candidate parent–child relationships:

$$\mathcal{L}_{edge} = \text{BCE}(\hat{E}, E). \quad (2)$$

This loss penalizes spurious dependencies while encouraging the reconstruction of valid execution paths, preserving the structural integrity of synthetic traces.

3.4.3 Latent Regularization. Latent variables are regularized using Kullback–Leibler (KL) divergence with respect to a standard Gaussian prior. Separate regularization terms are applied to graph-level and node-level latent variables.

Formally, the regularization terms are defined as:

$$\mathcal{L}_{KL} = \beta_G \text{KL}(q(z_G | G) \| \mathcal{N}(0, I)) + \beta_N \sum_{v \in V} \text{KL}(q(z_v | G) \| \mathcal{N}(0, I)), \quad (3)$$

where β_G and β_N control the strength of regularization for graph-level and node-level latent variables, respectively. In practice, we set $\beta_G > \beta_N$ to encourage stable modeling of global execution structure while allowing greater flexibility in local span-level variation.

To improve training stability, KL annealing is applied by gradually increasing β_G and β_N from zero during early training epochs. This allows the model to prioritize reconstruction accuracy before enforcing latent regularization.

3.4.4 Overall Objective. The full training objective is given by:

$$\mathcal{L}_{total} = \mathcal{L}_{node} + \lambda_{edge} \mathcal{L}_{edge} + \mathcal{L}_{KL}, \quad (4)$$

where λ_{edge} balances the contribution of execution structure reconstruction relative to node-level reconstruction. This objective encourages the model to learn latent representations that jointly preserve structural fidelity, semantic correctness, and controlled variability, which are all essential for generating useful synthetic distributed traces.

3.5 Synthetic Trace Generation

After training, synthetic distributed traces are generated by sampling from the learned latent space and decoding these samples into complete execution graphs. During this phase, the encoder is no longer required; trace synthesis relies exclusively on the decoder and the learned latent distributions. This separation ensures that synthetic trace generation does not depend on access to real execution data, which is critical for privacy-preserving deployment and offline data sharing.

Formally, synthetic trace generation proceeds by sampling a graph-level latent variable

$$z_G \sim \mathcal{N}(0, I),$$

and a set of node-level latent variables

$$\{z_v\}_{v=1}^{|V|}, \quad z_v \sim \mathcal{N}(0, I),$$

where $|V|$ denotes the number of spans in the generated trace. These latent variables are passed to the decoder, which reconstructs node attributes and execution structure to produce a synthetic trace graph $\hat{G} = (\hat{V}, \hat{E})$.

Since generation lacks skip connections, the model relies solely on latent variables. This forces z_G and $\{z_v\}$ to capture sufficient

global structure and local variability, enabling robust standalone synthesis without memorization.

3.5.1 Fixed-Size and Variable-Size Generation. To support different downstream analysis requirements, the framework supports two trace generation modes.

In *fixed-size generation*, the number of nodes $|V|$ is held constant and matched to a target trace size. This setting isolates the quality of node attribute generation and execution dependency reconstruction by removing graph-size variability. Fixed-size traces are particularly useful for controlled fidelity analysis and ablation studies.

In *variable-size generation*, the number of nodes $|V|$ is sampled from an empirical distribution derived from observed trace sizes. This setting evaluates the model’s ability to generate structurally coherent traces across heterogeneous execution depths and workloads. Variable-size generation introduces controlled structural diversity, which is important for training robust downstream models.

Both generation modes are fully supported by the same trained decoder and differ only in how the node count is determined at generation time.

3.5.2 Graph Construction and Thresholding. To construct a discrete execution graph from decoder outputs, directed edge logits representing parent–child dependency probabilities are produced for ordered node pairs. These logits are converted into binary edges using a fixed threshold, yielding the final execution edge set $\hat{E}$. This thresholding step controls graph sparsity and prevents the generation of unrealistically dense execution graphs.

The directed acyclic structure of distributed traces is preserved by restricting candidate edges to respect causal ordering. While edge prediction has quadratic complexity in the number of nodes, generated trace sizes remain within practical bounds for typical microservice workloads.

Each synthetic dataset contains the same number of traces as its real counterpart, enabling direct dataset-level comparison during evaluation. Importantly, synthetic trace generation is fully decoupled from the original datasets and can be repeated to produce arbitrarily large trace collections once training is complete.

4 Evaluation

Our evaluation assesses whether synthetic traces are sufficiently realistic for industrial analysis, prioritizing task-relevant fidelity over adversarial indistinguishability. Informed by discussions with engineers at Ciena regarding privacy and data scarcity, we aim to enable model development without exposing sensitive production data.

To ensure reproducibility while avoiding proprietary constraints, we validate our approach using public datasets that mimic industrial workloads. Following established practices [4, 8], we evaluate statistical similarity and downstream utility, including reconstruction fidelity, train-on-synthetic-test-on-real (TSTR), and hybrid training. This focuses on practical robustness for AIOps pipelines rather than exact replication. All source code is publicly available.¹

¹https://github.com/sneh2001patel/distributed_trace_research

4.1 RQ1: Fidelity of Synthetic Traces

This research question examines how accurately synthetic traces preserve the structural and feature-level properties of real distributed traces. Addressing **RQ1** is a prerequisite for practical deployment: if synthetic traces fail to capture essential execution semantics or structural dependencies, they cannot be trusted for downstream analysis or learning.

Reconstruction Fidelity. We first evaluate encoder–decoder reconstruction fidelity to assess whether the hierarchical VAE learns latent representations that preserve key properties of distributed traces. Reconstruction fidelity is not an end goal in itself, but a necessary validation step to ensure that the learned representations capture meaningful execution behavior before being used for synthetic trace generation.

Table 1 reports reconstruction performance for both datasets. For the SocialNetwork (SN) dataset, the model achieves perfect reconstruction of service identifiers and 99.91% accuracy for operation identifiers. Execution structure reconstruction is strong, achieving an edge F1-score of 97.65%. Span duration reconstruction yields a mean absolute error (MAE) of 1,653,559 ms, reflecting the wide dynamic range and heavy-tailed execution times typical of SN workloads.

For the TrainTicket (TT) dataset, service and operation reconstruction accuracies remain above 99.9%. Duration reconstruction yields an MAE of 1,561 ms over a narrower execution range. Structural reconstruction achieves an edge F1-score of 86.10%, characterized by high recall (95.65%) and moderate precision (78.28%). This behavior reflects higher structural ambiguity and branching variability in TT traces rather than systematic reconstruction failure.

Interpretation. These results indicate that the encoder captures compact yet expressive representations of both node-level attributes and execution dependencies. High reconstruction fidelity demonstrates that the model preserves the information necessary to generate structurally valid and semantically meaningful traces, while not implying memorization of individual executions.

Edge-level precision, recall, and F1 directly validate the decoder’s ability to reconstruct directed execution dependencies, confirming that structural constraints imposed during generation are preserved in practice.

RQ1 Finding. **RQ1 is answered as follows:** the proposed hierarchical VAE preserves essential structural and feature-level properties of real distributed traces with high fidelity, providing a reliable foundation for synthetic trace generation without collapsing execution diversity.

4.2 RQ2: Downstream Utility and Generalization

This research question examines to what extent synthetic traces can replace or supplement real trace data in downstream analysis tasks. Addressing **RQ2** is critical for industrial adoption, where access to large volumes of labeled production traces is often constrained by privacy, compliance, and retention policies, as discussed with engineers at Ciena.

Metric	SN Value	TT Value
Total nodes evaluated	11,649	7,912
Service Accuracy	100%	99.90%
Operation accuracy	99.91%	99.92%
Duration MAE	1,653,559.13 ms	1,561.19 ms
Edge Precision	95.40%	78.28%
Edge Recall	100%	95.65%
Edge F1-score	97.65%	86.10%

Table 1: Reconstruction performance for the SN and TT dataset trained for 300 epochs

Although our empirical validation uses public benchmark traces, the evaluated properties—structural fidelity, generalization under distribution shift, and robustness to trace variability—are model- and dataset-agnostic. These properties directly reflect requirements observed in industrial microservice deployments, where trace-driven models must generalize across evolving workloads and limited data availability.

To evaluate downstream utility under realistic deployment conditions, we assess classification performance on unseen workloads that differ from the traces used during training. Specifically, we use additional trace datasets released by the original dataset authors [14], which represent alternative executions and configurations of the same microservice systems. Neither the generative model nor the downstream classifier is retrained or fine-tuned on these datasets. This setup mirrors industrial scenarios in which models trained on historical traces must generalize to evolving system behavior without continuous retraining on production data.

Synthetic-Only Training. We first consider a synthetic-only training setting, in which the downstream classifier is trained exclusively on synthetic traces and evaluated on unseen real traces. Under this setting, classification accuracy remains well above random chance across all datasets. For the SocialNetwork system, accuracy ranges from approximately 75% to 78%, while for the TrainTicket system, accuracy ranges from approximately 72% to 76%.

Although performance degrades relative to in-distribution evaluation, these results demonstrate that synthetic traces preserve workload-discriminative structure that transfers beyond the specific executions observed during training. The classifier remains effective despite distributional shift, indicating that the generated traces capture stable execution patterns rather than dataset-specific artifacts.

Hybrid Training with Limited Real Data. We next evaluate a hybrid training setting in which a small fraction of real traces is combined with synthetic data during training. This setting reflects common industrial practice, where limited production traces may be available but cannot be broadly shared or retained at scale. Table 2 reports detailed classification metrics under hybrid training, including accuracy, precision, recall, and error counts.

With hybrid training, downstream performance improves substantially. Classification accuracy reaches 99.2%–100% across all unseen datasets, with misclassifications reduced to single-digit counts or eliminated entirely. These results indicate that synthetic traces capture the majority of task-relevant structure, while limited

Metric	Fixed-Size Dataset	Variable-Size Dataset
Test Loss	0.0121	0.0136
Test Accuracy	99.8%	99.9%
SN Accuracy	99.6%	99.7%
TT Accuracy	100%	100%
SN Precision / Recall / F1	1.000 / 0.996 / 0.998	1.000 / 0.997 / 0.999
TT Precision / Recall / F1	0.996 / 1.000 / 0.998	0.997 / 1.000 / 0.999
False Positives (SN → TT)	4	3
False Negatives (TT → SN)	0	0

Table 2: Classification performance with 10% real data + synthetic data

real data primarily serves to anchor the learned decision boundary to the true data manifold.

From a deployment perspective, this finding is particularly important. It suggests that synthetic traces can be used to bootstrap learning-based observability tools and substantially reduce reliance on sensitive production traces. This aligns with industrial constraints identified in discussions with Ciena, where privacy, compliance, and data availability limit the feasibility of large-scale trace collection for model development and validation.

RQ2 Finding. RQ2 is answered as follows: synthetic traces can effectively replace real traces for initial model training and can substantially reduce the amount of real data required to achieve high downstream performance, particularly when combined with a small fraction of production traces under distributional shift.

4.3 RQ3: Effect of Trace Variability on Robustness

This research question examines how structural variability in synthetic traces affects downstream robustness. In particular, **RQ3** compares fixed-size trace generation, which enforces uniform graph structure, with variable-size generation, which introduces controlled variability in execution depth and span count. Understanding this trade-off is important for industrial deployment, where real systems exhibit heterogeneous execution patterns rather than fixed structural templates.

To address this question, we evaluate downstream classification performance using models trained exclusively on synthetic traces generated under fixed-size and variable-size settings. This isolates the impact of structural variability without confounding effects from real data.

Fixed-Size versus Variable-Size Training. Table 3 summarizes classification performance for both generation regimes. Overall accuracy is comparable across settings (82.6% for fixed-size and 83.4% for variable-size generation), indicating that both approaches preserve workload-discriminative information.

However, differences emerge at the class level. Fixed-size generation yields higher recall for SocialNetwork traces, reflecting conservative reconstruction of common execution structures. In contrast, variable-size generation improves recall for TrainTicket traces and yields more balanced precision and recall across workloads. As a result, variable-size generation achieves higher macro-F1 and balanced accuracy.

Interpretation. These results highlight a trade-off between structural regularity and representational expressiveness. Fixed-size traces promote stability by reinforcing frequent execution patterns, whereas variable-size traces expose learning algorithms to a broader range of execution depths and branching behaviors. This exposure improves robustness under workload heterogeneity, which is characteristic of production microservice systems.

RQ3 Finding. RQ3 is answered as follows: introducing controlled trace variability through variable-size generation improves downstream robustness and class balance, while maintaining comparable overall accuracy to fixed-size generation. This suggests that variable-size synthetic traces are better suited for training robust analysis models in heterogeneous industrial environments.

4.4 RQ4: Similarity and Separability of Synthetic Traces

To assess dataset-level realism, we compare real and synthetic traces using alignment-free structural statistics and feature-space overlap.

Structural and Statistical Similarity. Instance-level similarity analysis reveals substantial variability across synthetic samples. Most synthetic traces exhibit partial rather than near-duplicate overlap with real executions, indicating that the generator avoids memorization while preserving task-relevant structural patterns.

Dataset-level KS tests confirm that coarse structural constraints, such as node count, are preserved exactly where enforced. Statistically significant deviations in higher-order structural metrics reflect controlled variability rather than reconstruction failure, consistent with a generative model that prioritizes realism and diversity over exact distributional replay.

Table 4 summarizes instance-level similarity statistics between synthetic and real traces.

We first evaluate distribution-level alignment using Kolmogorov–Smirnov (KS) tests over global structural metrics, which assess whether synthetic traces reproduce coarse-grained execution properties without enforcing instance-level correspondence. Table 5 reports Kolmogorov–Smirnov (KS) test results comparing global structural distributions between real and synthetic traces under fixed-size generation. Exact agreement in node count distributions ($KS = 0.000, p = 1.0$) confirms that graph-size constraints are strictly enforced. In contrast, statistically significant differences are observed for higher-order structural metrics such as edge count, density, average degree, and clustering coefficient. These deviations indicate that while coarse structural properties are preserved, the generator intentionally allows variability in more complex structural correlations, avoiding distribution-level memorization while maintaining realistic execution structure.

While KS tests capture marginal distribution alignment, they do not assess whether real and synthetic traces occupy overlapping regions of the joint feature space. We therefore complement this analysis with unsupervised clustering.

Observed differences in higher-order structural metrics partly reflect the use of probabilistic edge thresholding during generation, which intentionally avoids over-dense graphs and promotes structural diversity rather than exact distributional replay.

Dataset	Acc $\uparrow$	SN Prec	SN Rec	SN F1	TT Prec	TT Rec	TT F1
Fixed-Size	0.826	0.788	0.893	0.837	0.877	0.760	0.814
Variable-Size	0.834	0.868	0.787	0.825	0.805	0.880	0.841

Table 3: Binary classification performance (SN vs. TT) on synthetic datasets. Metrics are reported on held-out test sets.

Metric	Variable-Size		Fixed-Size	
	SN	TT	SN	TT
Average Overall Similarity	20.60%	26.22%	30.51%	63.90%
Max Observed Similarity	66.06%	60.89%	86.11%	100.00%
Min Observed Similarity	13.03%	10.14%	21.88%	22.98%
Average Max Similarity (nearest real)	26.26%	39.20%	—	—
Average Min Similarity	17.01%	15.78%	—	—
$\geq 80\%$ Overall Similarity	0/1244 (0.0%)	0/1244 (0.0%)	1/1244 (0.1%)	509/1244 (40.7%)
50–80% Overall Similarity	8/1244 (0.6%)	58/1244 (4.7%)	38/1244 (3.1%)	100/1244 (9.5%)
$< 50\%$ Overall Similarity	1236/1244 (99.4%)	1186/1244 (95.3%)	1205/1244 (96.9%)	635/1244 (49.8%)
$\geq 80\%$ Max Similarity	0/1244 (0.0%)	51/1244 (4.1%)	—	—
50–80% Max Similarity	12/1244 (1.0%)	129/1244 (10.4%)	—	—

Table 4: Instance-level similarity statistics between synthetic and real traces for variable-size and fixed-size generation. Similarity is computed using alignment-free structural metrics.

Structural Metric	SN KS	SN p-value	TT KS	TT p-value
Nodes	0.000	1.0	0.000	1.0
Edges	0.785	0.0	0.316	2.95e-55
Density	0.999	0.0	0.566	4.85e-184
Average Degree	1.000	1.0	0.590	3.24e-201
Average Clustering	0.991	1.0	0.521	2.00e-154

Table 5: Dataset-level distribution similarity (KS Test)

Unsupervised Separability. Unsupervised K-means clustering reveals substantial mixing between real and synthetic samples across both workloads and generation regimes. Low normalized mutual information (NMI) scores indicate limited separability in the dominant feature space [2], with deviations primarily associated with rare or long-tail behaviors.

PCA visualizations of node-level features (Figures 2–5) show similar behavior. Real and synthetic samples occupy overlapping regions, with deviations concentrated in rare or long-tail behaviors. This is consistent across fixed-size and variable-size settings. These visualizations corroborate the low NMI scores observed in Table 6, providing qualitative evidence that synthetic traces align closely with real traces in the dominant feature space. The observed overlap in both global structural statistics and node-level feature space is consistent with the hierarchical latent design of the proposed model. Graph-level latent variables constrain overall execution structure, while node-level latent variables introduce controlled local variability, which is desirable for training and benchmarking learning-based observability tools. Together, these mechanisms promote alignment with real execution behavior without collapsing diversity or memorizing individual traces.

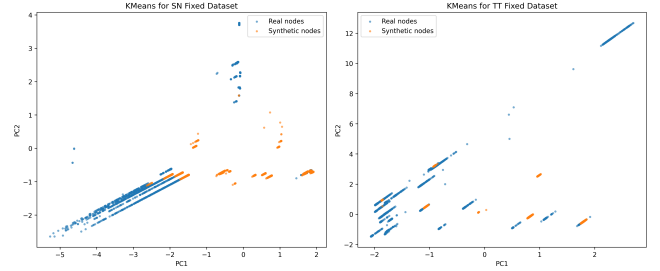


Figure 2: Fixed dataset SN PCA

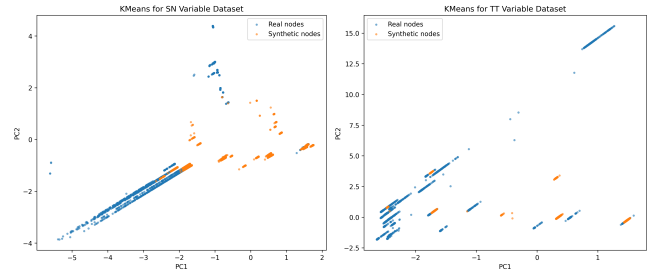


Figure 3: Fixed dataset TT PCA

Figure 4: Variable dataset SN PCA

Figure 5: Variable dataset TT PCA

Supervised Separability. Supervised RandomForest classifiers distinguish traces primarily via fine-grained artifacts; controlling these reduces performance to near chance. This highlights a trade-off: strict indistinguishability often collapses diversity. Consequently, we prioritize task-relevant similarity and diversity over adversarial

Dataset	Setting	Feature Dim.	Cluster	Cluster Size	Frac. Synthetic	NMI ↓
TT	Fixed-Size	30	C0	14,829	0.534	0.098
			C1	995	0.000	0.098
SN	Fixed-Size	72	C0	1,033	0.001	0.072
			C1	22,265	0.523	0.072
TT	Variable-Size	3	C0	995	0.000	0.117
			C1	21899	0.684	0.117
SN	Variable-Size	72	C0	17193	0.831	0.142
			C1	22,265	0.417	0.142

Table 6: KMeans clustering analysis of real and synthetic traces using node-attribute feature vectors (one-hot service ID, one-hot operation ID, and scaled duration). Cluster mixing reports the fraction of synthetic samples per cluster. NMI measures separability between real and synthetic labels (lower indicates stronger mixing).

indistinguishability, accepting minor artifacts to support robust downstream learning.

RQ4 Finding. RQ4 is answered as follows: synthetic traces exhibit strong overlap with real traces in the joint feature space and are difficult to distinguish using common unsupervised analysis techniques, while remaining sufficiently diverse to avoid memorization and support robust downstream learning.

Taken together, the evaluation shows that the proposed hierarchical VAE captures the structural and semantic characteristics of distributed traces in a way that supports downstream analysis. Across RQ1 and RQ4, reconstruction fidelity, structural statistics, clustering, and feature-space analyses indicate strong alignment between real and synthetic traces without evidence of memorization. The model preserves causal execution structure and span-level semantics while allowing controlled variation in higher-order structural properties, reflecting realistic behavior rather than exact distributional replay. This balance enables the generation of traces that are structurally valid and representative, yet not tied to specific execution instances.

From a learning and deployment perspective, results from RQ2 and RQ3 show that synthetic traces provide practical value beyond surface-level similarity. Models trained on synthetic data generalize to unseen real traces under distribution shift, and combining synthetic traces with a small fraction of real data yields near-optimal downstream performance. In addition, variable-size generation leads to more balanced classification behavior than fixed-size generation, indicating that structural variability affects robustness across workloads. These results collectively demonstrate that synthetic traces can support effective model training under data scarcity while remaining sensitive to design choices in trace generation.

5 Industrial Deployment and Lessons Learned

This section discusses how synthetic distributed traces from our framework can be used in practice and summarizes lessons that follow from the empirical results.

5.1 Deployment Scenarios in Industrial Pipelines

Based on our evaluation, we identify several realistic scenarios in which synthetic distributed traces can provide tangible value in industrial settings.

Cold-start training and data scarcity. When trace-driven tools are deployed before sufficient production traces have been collected or retained, synthetic traces can be used to bootstrap models. Our synthetic-only experiments show that such models achieve non-trivial generalization to real traces (RQ2).

Privacy-preserving data sharing and collaboration. Production traces often encode sensitive operational details that cannot be shared across teams or organizations. Synthetic traces preserve structural and behavioral characteristics without reproducing exact executions, which enables safer sharing for benchmarking and collaborative development.

Testing and benchmarking of AIOps pipelines. Synthetic traces provide controllable yet realistic workloads for testing observability and AIOps pipelines. Variable-size generation is useful for evaluating robustness to heterogeneous execution depths and branching patterns that may be underrepresented in limited real datasets.

These scenarios highlight that synthetic traces are not intended to fully replace production tracing, but rather to complement it in settings where real data is scarce, sensitive, or costly to obtain.

5.2 Lessons Learned

Our empirical study yields several lessons that are broadly applicable to the use of synthetic traces in industrial contexts.

Lesson 1: Perfect indistinguishability is not a practical objective. While it is tempting to aim for synthetic traces that are indistinguishable from real traces under powerful discriminative models, our results show that enforcing such indistinguishability can suppress meaningful variability and introduce unintended bias. In practice, a limited degree of separability under high-capacity classifiers is acceptable—and often desirable—if synthetic traces preserve task-relevant structure and support downstream learning.

Lesson 2: Structural variability improves downstream robustness. Allowing synthetic traces to vary in size and structure leads to more balanced downstream performance compared to fixed-size generation. Variable-size traces expose learning algorithms to a wider range of execution patterns, reducing overfitting to rigid structural templates and improving robustness across workloads.

Lesson 3: Small amounts of real data provide strong anchoring. We observe that augmenting synthetic training data with as little as 5–10% real traces is sufficient to achieve near-perfect downstream classification performance.

Lesson 4: Fidelity and diversity must be balanced based on the target task. High-fidelity reconstruction is important for tasks such as replay-based debugging or fine-grained performance analysis, whereas diversity is more valuable for training and stress-testing learning-based pipelines. Synthetic trace generation should therefore be configured based on the intended downstream use, rather than optimized for a single notion of realism.

5.3 Practical Guidelines for Practitioners

Based on these observations, we derive the following practical guidelines for industrial adoption:

- Use synthetic traces to bootstrap learning-based tools when real trace data is limited or unavailable.
- Combine synthetic traces with a small fraction of real data whenever possible to improve alignment with production behavior.
- Prefer variable-size trace generation when robustness and generalization are priorities.
- Avoid over-optimizing for indistinguishability under discriminative models at the expense of diversity and coverage.

Together, these guidelines provide concrete direction for integrating synthetic trace generation into real-world observability and AIOps workflows under realistic operational constraints.

6 Threats to Validity

Internal validity. Our evaluation relies on specific modeling choices (e.g., hierarchical VAEs and GCN encoders). While alternative architectures may yield different quantitative results, our conclusions focus on qualitative trends and trade-offs rather than absolute performance.

External validity. Experiments are conducted on two widely used microservice benchmarks (SN and TT). Although these systems differ in scale and structure, results may not fully generalize to all production workloads or tracing configurations.

Construct validity. We use established evaluation practices for synthetic data, including TSTR, clustering, and discriminative analysis. While no single metric fully captures realism, the combination of complementary analyses mitigates reliance on any one measure.

Conclusion validity. Statistical comparisons are based on empirical distributions rather than hypothesis testing across repeated deployments. Nevertheless, consistency across datasets, evaluation settings, and industry-informed deployment scenarios strengthens confidence in the observed trends.

7 Conclusion and Future Work

This paper presented a hierarchical VAE-based framework for generating synthetic distributed traces represented as graphs. Our evaluation shows that the generated traces preserve key structural and feature-level characteristics of real traces and exhibit strong overlap in the joint feature space. Through downstream train-on-synthetic-test-on-real experiments, we demonstrated that models trained on synthetic traces generalize effectively to real-world data, and that combining synthetic traces with a small fraction of real traces yields near-optimal performance.

Across both fixed-size and variable-size generation settings, clustering- and classifier-based analyses indicate that synthetic traces capture the underlying execution patterns of the original systems while introducing controlled variability. This variability enables the generation of a substantially larger and more diverse set of training samples than is typically available from production traces, which often contain limited structural diversity. As a result, synthetic data provides a practical mechanism for mitigating data scarcity and supporting robust learning-based analysis.

Importantly, our findings highlight that synthetic trace generation involves trade-offs between fidelity, diversity, and distinguishability. As discussed in Section 5, perfect indistinguishability from real traces is neither necessary nor desirable for many industrial use cases. Instead, balancing structural realism with variability and anchoring synthetic data using limited real traces emerges as an effective strategy for practical deployment.

While our study focuses on specific workloads and feature representations, the consistency of results across datasets, generation settings, and evaluation methods suggests that the proposed approach generalizes beyond the evaluated systems. Overall, this work demonstrates that hierarchical generative modeling provides a viable and scalable foundation for privacy-aware distributed trace synthesis, supported by empirical evidence across fidelity, generalization, and deployment-oriented evaluation scenarios, with clear applicability to real-world observability and AIOps pipelines.

Several directions for future work remain. First, while our study focuses on graph-structured traces with span-level attributes, extending the framework to incorporate richer temporal dynamics, resource metrics, or cross-trace dependencies could further improve realism for performance diagnosis tasks. Second, exploring alternative generative architectures, such as diffusion-based or flow-based models, may offer different trade-offs between fidelity, diversity, and training stability. Third, while our evaluation emphasizes offline analysis and benchmarking, integrating synthetic trace generation into online or continual learning pipelines remains an open challenge. Finally, large-scale deployment on proprietary production traces, in collaboration with industry partners, will enable deeper validation of the framework under real operational constraints and evolving workloads.

References

- [1] Laith Alzubaidi, Jinshuai Bai, Aiman Al-Sabaawi, Jose Santamaria, Ahmed Shihab Albahri, Bashar Sami Nayyef Al-Dabbagh, Mohammed A Fadel, Mohamed Manoufali, Jinglan Zhang, Ali H Al-Timemy, et al. 2023. A survey on deep learning tools dealing with data scarcity: definitions, challenges, solutions, tips, and applications. *Journal of Big Data* 10, 1 (2023), 46.
- [2] Olatz Arbelaitz, Ibai Gurrutxaga, Javier Muguerza, Jesús M Pérez, and Iñigo Perona. 2013. An extensive comparative study of cluster validity indices. *Pattern*

- recognition 46, 1 (2013), 243–256.
- [3] Dominik Ernst and Stefan Tai. 2021. Offline trace generation for microservice observability. In *2021 IEEE 25th International Enterprise Distributed Object Computing Workshop (EDOCW)*. IEEE, 308–317.
- [4] Cristóbal Esteban, Stephanie L Hyland, and Gunnar Rättsch. 2017. Real-valued (medical) time series generation with recurrent conditional gans. *arXiv preprint arXiv:1706.02633* (2017).
- [5] Rafael Estepa, Jesús E. Díaz-Verdejo, Antonio Estepa, and German Madinabeitia. 2020. How Much Training Data is Enough? A Case Study for HTTP Anomaly-Based Intrusion Detection. *IEEE Access* 8 (2020), 44410–44425. doi:10.1109/ACCESS.2020.2977591
- [6] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.
- [7] Aceto Giuseppe, Fabio Giampaolo, Ciro Guida, Stefano Izzo, Antonio Pescape, Francesco Piccialli, and Edoardo Prezioso. 2024. Synthetic and privacy-preserving traffic trace generation using generative ai models for training network intrusion detection systems. *Available at SSRN 4669010* (2024).
- [8] Andre Goncalves, Priyadip Ray, Braden Soper, Jennifer Stevens, Linda Coyle, and Ana Paula Sales. 2020. Generation and evaluation of synthetic patient data. *BMC medical research methodology* 20, 1 (2020), 108.
- [9] William L Hamilton. 2020. *Graph representation learning*. Morgan & Claypool Publishers.
- [10] Madeline Janecek, Naser Ezzati-Jivan, and Abdelwahab Hamou-Lhadj. 2025. Execution Trace Reconstruction Using Diffusion-Based Generative Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 600–600.
- [11] Donghyun Kim, Sriram Ravula, Taemin Ha, Alex Dimakis, Daehyeok Kim, and Aditya Akella. 2025. Large language models as realistic microservice trace generators. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 73–91.
- [12] Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).
- [13] TN Kipf. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [14] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1750–1762. doi:10.1109/ICSE48619.2023.00150
- [15] Bowen Li, Xin Peng, Qilin Xiang, Hanzhang Wang, Tao Xie, Jun Sun, and Xuanzhe Liu. 2022. Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empirical Software Engineering* 27, 1 (2022), 25.
- [16] Yingzhou Lu, Lulu Chen, Yuanyuan Zhang, Minjie Shen, Huazheng Wang, Xiao Wang, Capucine van Rechem, Tianfan Fu, and Wenqi Wei. 2023. Machine learning for synthetic data generation: a review. *arXiv preprint arXiv:2302.04062* (2023).
- [17] Lars Maaløe, Casper Kaae Sønderby, Søren Kaae Sønderby, and Ole Winther. 2016. Auxiliary deep generative models. In *International conference on machine learning*. PMLR, 1445–1453.
- [18] Lu Pang and Krishna Kant. 2026. Synthetic Data Generation for Storage Trace Augmentation. *ACM Transactions on Storage* 22, 1 (2026), 1–29.
- [19] Rajesh Ranganath, Dustin Tran, and David Blei. 2016. Hierarchical variational models. In *International conference on machine learning*. PMLR, 324–333.
- [20] Raja R Sambasivan. 2013. *Diagnosing performance changes in distributed systems by comparing request flows*. Ph. D. Dissertation. Carnegie Mellon University.
- [21] Benjamin H Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. 2010. Dapper, a large-scale distributed systems tracing infrastructure. (2010).
- [22] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*. 323–324.